Тестирование Java-приложений

JUnit, тестовые двойники и Mockito



Проверяем сервис и HTTP API задач

Мы уже проверили API через curl

- Добавили новую операцию.
- Изменили сервис и случайно сломали старую.
- Кто повторит все вчерашние проверки?

Автоматический тест сохраняет проверку и позволяет повторять её.

Что такое автоматический тест

- Создаёт известные исходные условия.
- Выполняет действие в программе.
- Сравнивает результат с ожидаемым.
- Сообщает об успехе или расхождении.

Тест содержит исполняемое ожидание от программы.

Разный масштаб проверки

Уровень	Пример для API задач
Unit	Правило завершения задачи в сервисе
Интеграционный	Контроллер вместе с MVC и JSON
Сквозной, E2E	Запущенное приложение и его реальные зависимости

Масштаб определяется границей проверки, а не именем аннотации.

Пирамида тестирования



Выше: шире сценарий и обычно дороже запуск

Много быстрых проверок, меньше дорогих сквозных сценариев.

Что даёт хороший набор тестов

- Быстро замечает изменение ожидаемого поведения.
- Позволяет увереннее менять внутреннее устройство кода.
- Показывает примеры использования и граничные случаи.
- Помогает локализовать причину ошибки.

Тесты снижают риск регрессий, но не доказывают отсутствие всех ошибок.

Тестовые зависимости

pom.xml. Внутри dependencies

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

JUnit Jupiter запускает тесты.

Mockito создаёт тестовые двойники.

Версии согласует Spring Boot. Scope test ограничивает зависимость тестами.

Где находятся тесты

Структура Maven-проекта

```
src/main/java/ru/example/tasks/
  Task.java
  TaskService.java
  TaskRepository.java

src/test/java/ru/example/tasks/
  TaskServiceTest.java
  FakeTaskRepository.java
```

Основной код и тесты разделены.

Имя класса заканчивается на Test.

Тест запускается из IDE и из сборки Maven.

Знакомая модель задачи

Java. Task.java

```
public record Task(
        long id, String title, boolean done) {

    public Task completed() {
        return new Task(id, title, true);
    }
}
```

Сохраняем record из занятия по REST.

completed() возвращает новое значение.

Завершение меняет done и сохраняет id и title.

Первый тест

Java. Фрагмент тестового класса

```
@Test
void completedTaskPreservesIdAndTitle() {
    Task task = new Task(7, "Изучить JUnit", false);

    Task actual = task.completed();

    assertEquals(
        new Task(7, "Изучить JUnit", true), actual);
}
```

@Test отмечает тестовый метод.

assertEquals: сначала ожидаемое, затем фактическое.

Сравниваем весь результат, включая поля, которые должны сохраниться.

Arrange — Act — Assert

Этап	В нашем тесте
Arrange: подготовка	Создать незавершённую Task
Act: действие	Вызвать completed()
Assert: проверка	Сравнить с ожидаемой Task

Один тест описывает один понятный сценарий.

Основные проверки JUnit

Проверка	Что выражает
assertEquals(expected, actual)	Равенство значений
assertTrue(condition)	Условие истинно
assertFalse(condition)	Условие ложно
assertThrows(Type.class, action)	Ожидаемое исключение

Проверяем результат автоматически, без чтения println.

Поймает ли тест ошибку?

- В completed() заменим true на false.
- А если случайно заменим title на пустую строку?
- Какое ожидание нарушится в каждом случае?

Полезный тест должен падать при поломке проверяемого поведения.

Принципы хорошего unit-теста

- Быстрый: его удобно запускать после изменения.
- Независимый: не требует запуска другого теста.
- Повторяемый: одинаковые условия дают одинаковый результат.
- Самопроверяющийся: успех определяется assertions.

Читаемое имя и небольшая подготовка помогают понять причину сбоя.

Независимость на практике

- Каждый тест сам готовит нужные ему данные.
- Не используем общую изменяемую static-коллекцию.
- Не полагаемся на порядок методов.
- Время и случайность делаем управляемыми, если они влияют на результат.

Тест должен проходить отдельно и вместе со всем набором.

Общая подготовка: @BeforeEach

Java. Фрагмент тестового класса

```
private FakeTaskRepository repository;
private TaskService service;

@BeforeEach
void setUp() {
    repository = new FakeTaskRepository();
    service = new TaskService(repository);
}
```

Выполняется перед каждым @Test.

Каждый тест получает новое хранилище.

Общие объекты создаём заново, данные конкретного сценария задаём в тесте.

Какие сценарии выбирать

Сценарий	Ожидание
Завершение существующей задачи	done = true, остальные поля сохранены
Неизвестный ID	TaskNotFoundException
Повторное завершение	Задача остаётся завершённой
Изменение одной из двух задач	Вторая задача не меняется

Сценарии выводим из правил приложения.

Проверяем поведение

- Вызываем публичную операцию и проверяем её результат.
- Не копируем алгоритм реализации в ожидаемое значение.
- Не привязываемся к private-методам и внутренним коллекциям.
- Сохраняем тесты при рефакторинге, который не меняет контракт.

Ожидание должно быть понятным без чтения реализации.

Как управлять зависимостями

Заготовленный ответ, хранилище в памяти
Затем Mockito

Контракт хранилища для примера

Java. TaskRepository.java

```
public interface TaskRepository {
    Optional<Task> findById(long id);
    Task save(Task task);
}
```

Выделяем интерфейс из реализации хранения.

save сохраняет задачу с заданным ID.

Сервис зависит от операций, а не от конкретной Map.

Сервис чтения задачи

Java. Метод TaskService

```
public Task get(long id) {
    return repository.findById(id)
        .orElseThrow(() ->
            new TaskNotFoundException(id));
}
```

repository — final-поле, переданное в конструктор TaskService.

Сценарий отсутствия строки проверяем независимо от способа хранения.

Stub: заранее заданный ответ

Java. Тестовая заглушка

```
class MissingTaskStub implements TaskRepository {
    public Optional<Task> findById(long id) {
        return Optional.empty();
    }
    public Task save(Task task) {
        throw new UnsupportedOperationException();
    }
}
```

Для сценария «задачи нет» всегда возвращаем пустой Optional.

save здесь использовать не планируем.

Stub задаёт ответ зависимости для выбранного сценария.

Тест ожидаемой ошибки

Java. Фрагмент тестового класса

```
@Test
void throwsWhenTaskMissing() {
    TaskService service = new TaskService(
        new MissingTaskStub());

    assertThrows(TaskNotFoundException.class,
        () -> service.get(99));
}
```

Lambda передаёт действие JUnit.

Исключение — ожидаемый результат этого сценария.

Если сервис молча вернёт null, тест упадёт.

Fake: рабочее хранилище в памяти

Java. FakeTaskRepository.java

```
class FakeTaskRepository implements TaskRepository {
    private final Map<Long, Task> tasks =
        new HashMap<>();
    public Task save(Task task) {
        tasks.put(task.id(), task);
        return task;
    }
    public Optional<Task> findById(long id) {
        return Optional.ofNullable(tasks.get(id));
    }
}
```

save действительно меняет состояние.

findById читает сохранённое значение.

Fake реализует упрощённое, но работающее поведение зависимости.

Чтение с fake-репозиторием

Java. Фрагмент тестового класса

```
@Test
void returnsExistingTask() {
    var repository = new FakeTaskRepository();
    var expected = new Task(7, "Git", false);
    repository.save(expected);
    var service = new TaskService(repository);

    Task actual = service.get(7);

    assertEquals(expected, actual);
}
```

Подготовка явно показывает исходные данные.

Используем настоящую логику сервиса с простым хранилищем.

Операция завершения

Java. Метод TaskService

```
public Task complete(long id) {
    Task task = get(id);
    return repository.save(task.completed());
}
```

Сначала читаем задачу. Затем создаём завершённое значение. Сохраняем результат.

Внешний результат операции включает изменение состояния хранилища.

Проверяем изменение состояния

Java. Фрагмент тестового класса

```
@Test
void completesAndStoresTask() {
    var repository = new FakeTaskRepository();
    repository.save(new Task(7, "Git", false));
    var service = new TaskService(repository);

    Task actual = service.complete(7);

    assertEquals(new Task(7, "Git", true), actual);
    assertEquals(actual,
        repository.findById(7).orElseThrow());
}
```

Первый assert проверяет ответ.

Второй проверяет сохранённое значение.

Если убрать save, проверка хранилища обнаружит ошибку.

Stub, fake и mock

Двойник	Для чего используем
Stub	Выдать заготовленный ответ
Fake	Получить упрощённую рабочую реализацию
Mock	Проверить нужное взаимодействие с зависимостью

Роль двойника определяется тем, как он используется в тесте.

Что fake не доказывает

- Правильность SQL и схемы будущей базы данных.
- Поведение сетевых ошибок и транзакций.
- Совпадение всех особенностей настоящего хранилища.
- Корректность подключения компонентов в Spring.

Реальную интеграцию проверяем отдельными тестами.

Mockito и Spring-тесты

Настраиваем ответы зависимости
Проверяем важные вызовы

Mockito: ответ без отдельного класса


```
TaskRepository repository = mock(TaskRepository.class);
when(repository.findById(7L))
    .thenReturn(Optional.of(new Task(7, "Git", false)));
var service = new TaskService(repository);

Task actual = service.get(7);

assertEquals(new Task(7, "Git", false), actual);
```

mock создаёт двойник.

when / thenReturn задаёт ответ.

Здесь Mockito-объект играет роль stub.

verify: проверка взаимодействия

Java. Внутри @Test, после создания mock

```
when(repository.findById(7L))
    .thenReturn(Optional.of(new Task(7, "Git", false)));
when(repository.save(new Task(7, "Git", true)))
    .thenReturn(new Task(7, "Git", true));
var service = new TaskService(repository);

Task actual = service.complete(7);

assertEquals(new Task(7, "Git", true), actual);
verify(repository).save(new Task(7, "Git", true));
```

repository создан через mock в начале теста.

verify проверяет вызов save с нужным значением.

Вызов проверяем, когда он выражает значимый результат операции.

Чего mock не делает сам

- Не начинает хранить объекты после вызова save.
- Не выполняет настоящую реализацию зависимости.
- Не определяет, каким должен быть правильный ответ.
- Не делает тест полезным без проверки нашего кода.

when настраивает поведение. assert и verify проверяют ожидания.

Подмена зависимости в Spring

Аннотация	Что означает
@MockitoBean	Создать или заменить bean Mockito-двойником
@Autowired	Получить объект из тестового контекста
@SpringBootTest	Поднять контекст приложения для теста

В старых примерах встречается @MockBean. Здесь используем @MockitoBean.

Сервис из тестового Spring-контекста

Java. TaskServiceSpringTest.java

```
@SpringBootTest(classes = TaskService.class,
    webEnvironment = SpringBootTest.WebEnvironment.NONE)
class TaskServiceSpringTest {
    @MockitoBean
    TaskRepository repository;
    @Autowired
    TaskService service;
    // Тест на следующем слайде
}
```

Контекст ограничен нашим сервисом.

В его конструктор попадёт mock-репозиторий.

Проверяем настоящий TaskService, подменяем его зависимость.

Тест с @MockitoBean

Java. Фрагмент тестового класса

```
@Test
void returnsExistingTask() {
    when(repository.findById(7L))
        .thenReturn(Optional.of(new Task(7, "Git", false)));

    Task actual = service.get(7);

    assertEquals(new Task(7, "Git", false), actual);
}
```

Поля взяты из предыдущего слайда.

Логика теста почти не изменилась.

Теперь объекты связывает Spring, но обычному unit-тесту контекст не нужен.

Проверка контроллера отдельно

Java. TaskControllerTest.java

```
@WebMvcTest(TaskController.class)
class TaskControllerTest {
    @Autowired
    MockMvc mvc;
    @MockitoBean
    TaskService service;
    // Тест на следующем слайде
}
```

Настоящий контроллер и MVC.

Сервис заменён двойником.

MockMvc выполняет запрос через MVC без запуска сетевого сервера.

GET /tasks/7: статус и JSON

Java. Фрагмент тестового класса

```
@Test
void returnsTaskAsJson() throws Exception {
    when(service.get(7L))
        .thenReturn(new Task(7, "Git", false));

    mvc.perform(get("/tasks/7"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.id").value(7))
        .andExpect(jsonPath("$.title").value("Git"))
        .andExpect(jsonPath("$.done").value(false));
}
```

Проверяем маршрут, HTTP-статус и поля ответа.

Этот тест не выполняет настоящую логику сервиса и хранилища.

Выбор границы теста

Вопрос	Подход
Верно ли правило сервиса?	JUnit + new + stub / fake / mock
Верно ли работает HTTP-контракт?	WebMvcTest + MockMvc
Верно ли связаны реальные компоненты?	Интеграционный тест с нужными зависимостями
Работает ли основной путь целиком?	Небольшой набор сквозных проверок

Подмена зависимости ограничивает то, что доказывает тест.

Запуск в IDE и Maven

Терминал. Maven Wrapper

```
./mvnw test

./mvnw -Dtest=TaskServiceTest test

./mvnw -Dtest=TaskServiceTest#returnsExistingTask test
```

Запускаем весь набор, класс или один метод.

Отчёты: target/surefire-reports.

Перед передачей изменений запускаем весь набор тестов.

Покрытие и качество проверок

- Покрытие показывает, какой код выполнялся в тестах.
- Выполненная строка ещё не означает проверенный результат.
- Зелёный тест без содержательного assert мало что сообщает.
- Новая ошибка должна приводить к новому полезному сценарию.

Цель — замечать значимые поломки, а не только увеличивать процент.

Демонстрация после лекции

- Запускаем первый тест и намеренно ломаем completed().
- Проверяем сервис с fake, затем сценарий отсутствия со stub.
- Заменяем заглушку на Mockito и показываем verify.
- Запускаем один MVC-тест, затем весь набор через Maven.

После демонстрации вы добавите проверки самостоятельно.

Практика: проверки завершения задачи

- Неизвестный ID вызывает TaskNotFoundException.
- Повторное завершение сохраняет id, title и done = true.
- Завершение первой задачи не меняет вторую.
- Каждый тест проходит отдельно и вместе с остальными.

Базовое задание выполняется с FakeTaskRepository.

Какой двойник здесь нужен?

- Нужно всегда отвечать: «задачи нет».
- Нужно сохранить задачу, а потом прочитать её.
- Нужно проверить передачу завершённой задачи в save.

Stub, fake и mock решают разные задачи.

Материалы к занятию

- JUnit Jupiter: assertions и жизненный цикл тестов.
- Mockito: настройка ответов и проверка взаимодействий.
- Spring: MockitoBean и тестирование MVC.
- Пирамида тестирования и классификация тестовых двойников.

Ссылки находятся в заметках к слайдам.

Теперь запустим тесты

Сначала увидим, как тест ловит поломку
Затем напишем проверки сервиса и API